PocketChaos: Hackathon Submission Report

Mobile API Chaos Proxy & Negative-Testing Companion

FiftyFive Build With AI Hackathon 2026 · Solo submission by Rahul R Mahawar · October 2026

At a glance

Problem	Mobile apps are tested on fast office Wi-Fi, then meet slow 3G, intermittent HTTP 5xx and drifting API contracts in production. Reproducing those conditions today means certificate-based desktop proxies, backend changes, or mocks that never touch a network.
Product	A chaos proxy (Express + MongoDB), a web control plane (React) and a React Native companion app (Expo), in one monorepo.
Primary journey	Sign up → define an endpoint → set chaos rules → pair a phone with a 6-character key → run a resilience batch → read live telemetry on the web and payload verdicts on the phone.
Quality evidence	**707 automated tests passing** (backend 349 · web 125 · mobile 233), strict TypeScript everywhere, zero hardcoded secrets, live runs on the iOS simulator and in the browser, plus an Android emulator network check.
AI practice	A written engineering contract (`.ai/CONTEXT.md`), a product blueprint, three agent roles, and a 5-entry AI learning log that records every AI failure as *symptom* → *cause* → *fix* → *guard*.

1. Product

1.1 Problem statement

Mobile developers struggle to test how their apps handle real-world network failure: flaky 3G latency, unexpected 4xx/5xx status codes, dropped connections and corrupted JSON payloads. The usual options each have a cost:

- **Desktop SSL proxies:** a root certificate on every device, manual Wi-Fi proxy settings, hand-written rewrite rules.
- **Changing the backend:** feature flags or "fail sometimes" code inside a real service, which then has to be removed again.
- **Mocks in unit tests:** fast, but they never cross a network, so timeouts, retries and offline queues stay untested.

The result is that error paths, retry logic and parsers ship without ever meeting the failures they exist for.

1.2 Who it is for

Mobile and front-end engineers (React Native, Flutter, native Android/iOS) and QA engineers who need **reproducible** bad-network and bad-payload conditions, on a real device, without owning or modifying the backend.

1.3 What PocketChaos does

Chaos proxy (`/backend`). Every request to `/proxy/:projectKey/*` is matched to an endpoint the developer defined, then:

1. **Latency injection:** waits `delayMs` (0–10,000 ms, clamped again at runtime).
2. **Probabilistic failure:** with probability `failureRate`% returns `overrideStatusCode` (400–599, default 500) and `{ "chaosFault": true, "message": "Injected by PocketChaos" }`.
3. **Response:** serves the endpoint's mock JSON, or forwards to an SSRF-vetted upstream URL.
4. **Payload corruption:** if `corruptJson` is on, about 25% of the fields of a successful JSON body (1–10 of them) become `null`, change type or disappear, while the response keeps its 200 status.
5. **Telemetry:** writes a `RequestLog` (status, latency, chaos applied, client platform) and returns an `X-PocketChaos-Applied` header such as `delay:250ms,corrupt_json:3`.

Web control plane (`/web`).

- Registration and login.
- An endpoint manager for mock or forward mode.
- Live chaos controls: sliders and toggles, debounced and applied instantly via `PATCH`.
- **Test in Browser** for one-off proxy calls.
- A telemetry panel that polls every 2 s, colouring each request by outcome (green success, amber corrupted, red fault) and tagging the chaos applied.

Mobile companion (`/mobile`).

- **Pairing:** with a 6-character project key and server presets (LAN, Android emulator `10.0.2.2`, localhost, cloud, custom URL).
- **Batch Chaos Runner:**
- 5/10/20 requests per endpoint.
- Per-attempt timeouts of 2/4/8/12 s.
- 0–3 retries with exponential backoff, jitter and `Retry-After`.
- A live resilience summary.
- **Schema Integrity Detector:** learns each endpoint's JSON shape and flags corrupted payloads as `SCHEMA_CORRUPTION_DETECTED`, `INVALID_JSON` or `UNVERIFIED`.
- **Offline Queue & Sync Simulator:** with persistent, ordered, single-flight replay and one `Idempotency-Key` per request.
- **Payload Inspector:** shows headers, the retry timeline, deviations and the raw body.

1.4 Primary user journey

1. **Sign up** on the dashboard; the account receives a unique 6-character `projectKey`.

2. **Define an endpoint:** e.g. `GET /api/v1/user/profile`, with mock JSON or an upstream URL.
3. **Set chaos rules:** e.g. +2,500 ms latency, 40% → 503, corrupt JSON.
4. **Pair the phone:** type the key on the Pair tab. The phone never receives a password or JWT.
5. **Run a batch:** watch retries, timeouts, faults and corrupted payloads being classified; toggle **Simulate Offline Mode** to queue requests and replay them later.
6. **Read the results:** live telemetry on the web, per-request verdicts and raw payloads on the phone.

1.5 Scope

- **In scope:** JWT auth, endpoint CRUD, chaos rule engine, live request logs, the React Native test runner, the offline queue tester and the payload inspector.
- **Out of scope (deliberately):** system-level VPN/SSL interception, billing, multi-organisation RBAC, WebSocket streaming.

1.6 Running the demo

Step	Command
Backend (in-memory MongoDB, no install)	`cd backend && JWT_SECRET=$(openssl rand -hex 32) CORS_ORIGINS=http://localhost:5173 npm run dev:memory`
Web dashboard	`cd web && npm run dev` → http://localhost:5173
Demo data + live traffic	`node submission-assets/demo-seeder.ts --traffic`
Mobile app	`cd mobile && npx expo start` → Expo Go on a device or simulator

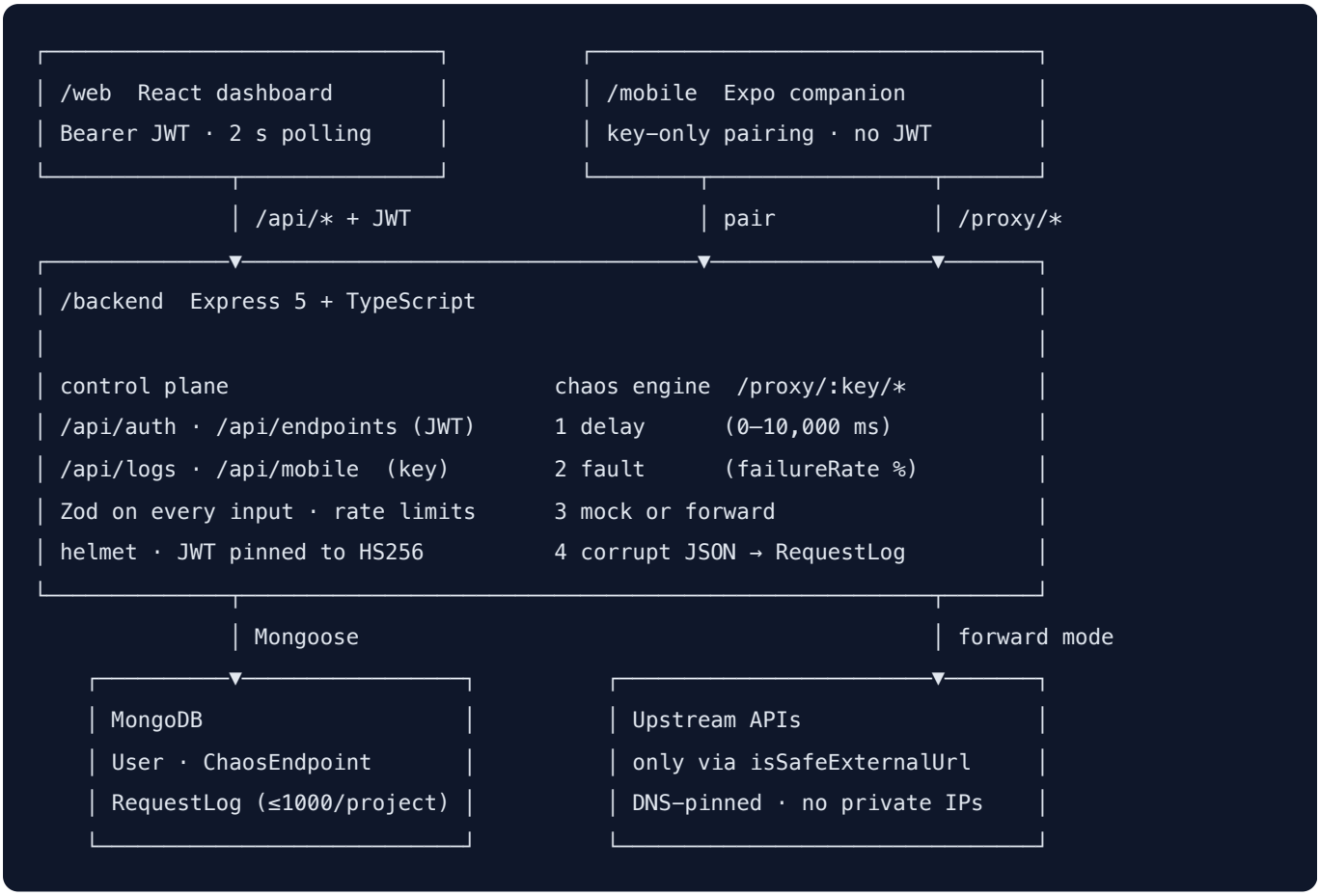
The demo seeder prints the project key in large type, creates three endpoints, one per chaos behaviour, and can stream simulated mobile traffic so the telemetry panel fills up during a recording.

Deployment status: the product runs end to end locally. The backend is deployment-ready in configuration terms: its environment is validated at startup, `TRUST_PROXY` handles reverse proxies, and CORS uses an explicit allow-list. If a hosted instance is provided, its access details are shared through the submission process, never in the repository.

---

2. Architecture & technical design

2.1 System overview



2.2 Technology stack

Area	Stack
Backend	Node.js ≥ 20.19, Express 5, TypeScript 5.9, Mongoose 9, Zod 4, jsonwebtoken (HS256), bcryptjs, helmet, express-rate-limit
Web	React 19, Vite 8, TypeScript 5.9, Tailwind CSS 4, Axios, Zod, lucide-react
Mobile	Expo SDK 58, React Native 0.88, TypeScript 6 (strict), React Navigation 7, AsyncStorage, Axios, Zod, expo-crypto
Testing	Jest 30 + Supertest + mongodb-memory-server (backend); Vitest 5 + Testing Library + jsdom (web); jest-expo + React Native Testing Library (mobile)

2.3 API contract

All `/api/*` responses use one envelope: `{ "success": true, "data": … }` or `{ "success": false, "error": { "code", "message", "details": [] } }`.

Route	Auth	Purpose
`POST /api/auth/register`, `POST /api/auth/login`	none (30 req / 15 min)	Returns a JWT and the account's `projectKey`
`GET` / `POST /api/endpoints`	Bearer JWT (300/min)	List or create the caller's endpoints (max 100)
`PATCH /api/endpoints/:id/chaos`	Bearer JWT, owner only	Change any subset of the chaos rules
`DELETE /api/endpoints/:id`	Bearer JWT, owner only	Delete an endpoint
`GET /api/logs/:projectKey`	project key (120/min)	Newest 50 request logs
`GET /api/mobile/pair/:projectKey`	project key (120/min)	Endpoints and chaos rules for the phone; hides upstream URLs and mock bodies
`ALL /proxy/:projectKey/*`	project key (600/min)	The chaos engine

2.4 Key technical decisions and trade-offs

Decision	Why	Cost we accepted
Zod-validated environment at startup	`JWT_SECRET` (≥ 32 chars), `MONGODB_URI` and `CORS_ORIGINS` have no defaults; a bad config stops the boot and names the key, never the value.	The server will not start until configuration is complete.
Short-polling (2 s) instead of WebSockets	Stateless, works behind any host or proxy, trivial to rate-limit; polling pauses in hidden tabs and backs off on 429.	Up to 2 s of staleness and some extra requests.
Deterministic AsyncStorage offline queue	One serialized writer, persisted sequence numbers, single-flight sync, in-flight markers, Zod-validated storage. Replay is ordered and never duplicated by a double tap.	At-least-once delivery after a crash, made safe by one `Idempotency-Key` per request, which the proxy forwards.
Key-only mobile pairing	The phone never holds credentials; the pairing view exposes no upstream URLs or mock bodies.	Anyone with the key can read that project's logs: 32^6 keys plus rate limits make guessing impractical.
Schema learned, not declared	No schema upload needed: clean responses form the baseline; with 100% corruption a strict-majority consensus is used, and fields the batch cannot agree on are reported UNVERIFIED rather than INTACT.	A 1-field body under 100% corruption cannot be judged reliably without one clean sample.
Mock or forward per endpoint	Teams can start before their backend exists, then point at the real API.	Forward mode needs strict SSRF protection (§2.5).
In-memory MongoDB dev mode	`npm run dev:memory` runs the full API without installing MongoDB.	Data is discarded on restart.

2.5 Security and guardrails

Guardrail	Implementation	Evidence
Zero hardcoded secrets	Secrets and URLs only in the environment; `.env.example` files only; client bundles inline only `VITE_*` / `EXPO_PUBLIC_*` values	11 env-validation tests; canary builds of the web and mobile bundles leaked 0 secrets
Authentication & ownership	Bearer JWT pinned to HS256; every endpoint query scoped to `req.user.id`; bcrypt (12 rounds) with a timing equaliser for unknown emails	`alg=none`, HS512, expired and wrong-secret tokens → 401; User B cannot read, toggle or delete User A's endpoints
Input validation	Zod middleware on every body, param and query before the controller; no type coercion; unknown keys rejected; mock JSON depth capped at 20	Boundary tests at 20 / 21 / 150 / 40,000 levels; NoSQL operator objects → 400
Bounded resources	`delayMs` 0–10,000 validated and re-clamped at runtime; upstream timeout 10 s and 2 MB cap; 100 kb request bodies; logs capped at 1,000 per project with a 7-day TTL; rate limits on every route group	`delayMs = 10000` sleeps exactly 10 s; a stored `999999` still sleeps 10 s; gzip bombs → `UPSTREAM_TOO_LARGE`
SSRF protection	`isSafeExternalUrl` / `checkExternalUrl`: http(s) only; blocks loopback, `0.0.0.0`, RFC 1918, link-local incl. `169.254.169.254`, private and IPv4-mapped IPv6; DNS pinned at connect time; redirects not followed; `VettedUrl` branded type makes skipping the check a compile error	65 SSRF unit tests; DNS rebinding → 403
Proxy response isolation	`Content-Security-Policy: default-src 'none'; frame-ancestors 'none'; sandbox` on every proxy response; upstream `Set-Cookie` dropped	Integration tests
Mobile hardening	Stored queue items re-validated (key pattern, no `..` / `%2e` path segments); pairing responses validated; bodies always rendered as text, never as HTML	Tampered-storage and malformed-pairing tests

2.6 Main dependencies

App	Runtime dependencies
Backend	express 5.3, mongoose 9.11, zod 4.6, jsonwebtoken 9, bcryptjs 3, helmet 8.3, express-rate-limit 8.7, cors 2.8
Web	react 19.3, react-dom 19.3, axios 1.20, zod 4.6, lucide-react 1.59
Mobile	expo ~58, react-native 0.88, @react-navigation 7, @react-native-async-storage 2.2, axios 1.20, zod 4.6, expo-crypto, expo-constants

`npm audit --omit=dev` reports 0 vulnerabilities for the backend and web. On mobile, `npm audit` reports 22 findings, all in Expo and Metro build tooling, none of which ships in the app bundle.

2.7 Known limitations and accepted risks

- **Dashboard JWT in `localStorage`:** readable by any XSS. React escapes all rendered data and no HTML sinks are used. A build-time CSP is the recommended next step.
- **Open redirect pass-through:** upstream `Location` headers are forwarded on purpose, so apps can test redirect handling.
- **Endpoint cap race:** the 100-endpoint cap is count-then-insert, so concurrent creates can slightly exceed it.
- **HEAD requests:** `HEAD` is not mapped to `GET` endpoints.
- **Corruption detection limit:** a single-field body under 100% corruption cannot be judged without a clean baseline (a clean request or an earlier batch provides one).
- **Development network settings:** cleartext HTTP is enabled for LAN development (`usesCleartextTraffic`, `NSAllowsLocalNetworking`) and should be disabled for a release build against an HTTPS backend.
- **Telemetry staleness:** short-polling means up to 2 s of telemetry staleness.

3. AI development workflow

3.1 AI across the development lifecycle

Stage	What AI did	Context / artefact	Human role
01 Define	Turned the idea into scope, data models, API contracts and a roadmap	`POCKETCHAOS_MASTER_PLAN.md`	Chose the problem, cut scope ("build smart, not big")
02 Design	Architect role confirmed Mongoose schemas, the SSRF strategy, web↔API communication, mobile base-URL handling and offline-queue design before code	`.ai/CONTEXT.md` (security rules, envelope, chaos semantics, standards), `.ai/agents/architect-agent.md`	Reviewed and accepted each design
03 Build	Implemented backend → web → mobile with Claude Code, phase by phase, against the contract	Phase prompts + CONTEXT.md	Ran every phase locally; verified on the iOS simulator and in the browser, plus an Android emulator network check
04 Test	QA-breaker role generated happy-path, boundary and negative suites; wrote a failing test for each suspected bug before fixing it	`.ai/agents/qa-breaker-agent.md`	Checked that tests failed for the right reason; broke code on purpose to prove tests bite

Stage	What AI did	Context / artefact	Human role
05 Review	Security-auditor role reviewed each phase for secrets, SSRF/injection, DoS and maintainability	`.ai/agents/sec-auditor-agent.md`	Decided fix vs. accepted risk
06 Document	Recorded every AI failure and fix	`.ai/AI_LEARNING_LOG.md` (5 entries)	Labelled each item *observed* or *review*; corrected over-claims

3.2 Context engineering (guidebook page 4)

Guidebook dimension	Where it lives
Requirements	`POCKETCHAOS_MASTER_PLAN.md` : problem, MVP scope, in/out of scope, user flows
Architecture	`.ai/CONTEXT.md` §2 (monorepo and stack), §5 (chaos engine semantics); master plan §3 (data models and API contracts)
Standards	`.ai/CONTEXT.md` §4 (response envelope), §6 (coding and testing standards)
AI context	`.ai/agents/*.md` (agent roles), `.ai/CONTEXT.md` §3 (mandatory security guardrails)
Delivery	`.env.example` in each app, `npm run dev:memory` , documented run commands, the submission tooling in `submission-assets/`
Learning	`.ai/AI_LEARNING_LOG.md` : five entries, one per phase or pass

3.3 Agents and instructions

Agent	Instruction	Output it produced
`architect-agent`	Review each proposed design against `.ai/CONTEXT.md` before code: exact TypeScript interfaces and Mongoose/Zod schemas; two trade-offs with the simplest reliable option for a solo MVP; three production failure modes and how to prevent them	Schema and SSRF plan (backend), web↔API communication design (web), base-URL and offline-queue design (mobile)
`qa-breaker-agent`	Four pillars: happy path; boundaries (`delayMs` 0/10000, `failureRate` 0/100, empty and deeply nested JSON); negative cases (malformed payloads, auth failures, SSRF attempts, unknown keys); bug report before tests	Backend, web and mobile test suites; bug discovery reports
`sec-auditor-agent`	Secrets and env leaks; SSRF and injection; resource exhaustion; maintainability; output as a Severity · File · Issue · Fix table	Proxy CSP sandbox, log cap, rate limits, mobile storage hardening

3.4 Guardrails we put on the AI

- **A written contract first.** `.ai/CONTEXT.md` states the security rules, response envelope and chaos semantics the model must follow.
- **Failing test first.** A suspected bug only counts once a test fails against the unfixed code. The fix must turn it green.
- **Count the tests.** A green run is evidence only if the test count is right.
- **Empty-file pre-flight.** Added after Entry #3: an empty context or agent file is a blocker, and `package-submission.sh` enforces it.
- **Verify before writing it down.** Limits and numbers in the log are measured, not assumed (see 1.3 and 5.13).

3.5 Learning from AI failures (selected)

Case (log ref)	What went wrong	How it was caught	What changed
SSRF guard blocked all public IPv4 (1.1)	One `net.BlockList` for both families; the IPv4-mapped IPv6 rule matched every IPv4 address	Allow-list tests (`8.8.8.8` must pass)	Per-family block lists; lesson: test the allow-list as hard as the block-list
Built against an empty contract (#3)	`CONTEXT.md` was 0 bytes on disk; the model filled the gap with its own defaults, producing 5 contract divergences	AI flagged the empty file; human review once it was saved	Code realigned; empty-file pre-flight added to the workflow
Jest "passed" with 0 tests (2.1)	A broken Watchman binary made Jest exit 0 without running anything	Test count check	`watchman: false`; rule: always read `Tests: N passed`
A test too weak to catch its bug (4.3)	A race test still passed with the protecting code removed	Deliberately broke the code	Test rewritten to assert the in-between state
Wrong "known limit" (1.3)	Cited MongoDB's nesting limit as 100 levels	Measured: 150 levels store fine; 180 fails	Depth cap kept, reason corrected; lesson: verify limits before writing them down
Offline queue erased itself (5.6)	After a failed storage read, the next enqueue overwrote every saved request	Failing test first	No write until a read succeeds; same bug class fixed in settings (5.11)
"INTACT" for corrupted payloads (5.13)	At 100% corruption the consensus detector called 16.8% of corrupted 4-field bodies INTACT	Measured against the backend's real corruptor over 1,000 batches	Undecided fields reported UNVERIFIED, which brought it down to 1.5%

4. QA and testing

4.1 Test suites

App	Tests	Tooling	Highlights
Backend	**349** (8 suites)	Jest 30, Supertest, mongodb-memory-server	endpoints 112 · proxy 75 · SSRF 65 · chaos service 33 · auth 22 · mobile & logs 18 · upstream 13 · env 11
Web	**125** (12 files)	Vitest 5, Testing Library	API client, form validation, telemetry formatting, polling, error boundaries
Mobile	**233** (15 suites)	jest-expo, RN Testing Library	integrity detector 42 + 20 QA, offline queue 15 + 28 QA, resilience 17 + 8 QA, screens and app-level races
Total	**707**, all passing		`npm test` in each app; TypeScript compiles clean in all three

4.2 AI-assisted QA approach

The `qa-breaker-agent` instruction drove each QA pass. The AI listed suspected bugs **before** writing tests, then wrote a failing test for each one, fixed the code, and re-ran the whole suite. Human validation checked that every red test failed *for the bug it targets*. One test-setup flaw was caught this way: a one-shot storage fault was consumed by the wrong caller, so the test was failing for a different reason. It was corrected and re-run red before the fix. One fix was re-proven by reverting it.

4.3 What the suites cover

Boundary cases:

- `delayMs` 0 / 10,000 / 10,001, plus a stored 999,999 that is clamped at runtime.
- `failureRate` 0 / 100.
- Override status 400–599.
- Mock JSON depth 20 (accepted) vs. 21 / 150 / 40,000 (rejected).
- Mobile timeout at 3,999 ms (success) vs. 4,000 ms (timeout), and a worst-case retry budget of exactly 12,900 ms.

Negative and attack cases:

- SSRF: loopback, RFC 1918, cloud metadata, IPv6, DNS rebinding, redirects.
- JWT tampering and cross-user access.
- NoSQL operator injection.
- Hostile mock keys (`__proto__`, `$where`).
- Malformed percent-encoding.
- 11 invalid project keys.
- 7 malformed JSON bodies from the proxy.
- Malformed pairing responses.

Chaos and failure cases:

- 100% corrupted batches.
- Replaying an empty queue.
- Same-frame double taps on Run and Replay.
- Storage read and write failures mid-sync.
- Offline mode toggled during a replay.
- 9,000-level nested JSON in the inspector.
- Unreachable servers and timeouts, which are never queued.

4.4 Running the tests

```
cd backend && npm test      # tsc + Jest (349)
cd web     && npm test      # Vitest (125)
cd mobile  && npm test      # jest-expo (233);  npx tsc --noEmit for types
```

5. Submission checklist (guidebook page 9)

Required	Where
Project description, problem statement	§1 of this report; slides 1–3 of `presentation-deck.html`
Working product / demo	§1.6 run commands; `demo-seeder.ts`; the demo video
ZIP with complete project and full Git history	`PocketChaos_Hackathon_Submission.zip`, built by `package-submission.sh` (includes `.git/`, excludes dependencies, builds and `.env` files)
Architecture, stack, decisions, limitations, dependencies	§2
How AI was used, workflow, context engineering, agents, instructions, guardrails	§3 and `.ai/`
AI-assisted QA approach	§4 and `.ai/AI_LEARNING_LOG.md`
Demo video (≤ 5 min)	Recorded from `VIDEO_PRODUCTION_GUIDE.md`
No secrets in repository or ZIP	Pre-flight secret scan in `package-submission.sh`; only `.env.example` files are tracked
Pre-existing work	None. The project was built during the hackathon (first commit 18 Oct 2026). Framework starter templates and the open-source dependencies in §2.6 are used under their licences.